In [1]:
```
import numpy as np
from scipy import stats
from sklearn.linear_model import LinearRegression
import plotly.express as px
import plotly.graph_objects as go

np.set_printoptions(precision=6, suppress=True)
```

In [2]:
```
def generate_dataset(n = 100, seed = 140):
    np.random.seed(seed)
    X1 = np.random.uniform(0,1,n)
    U = np.random.uniform(0,1,n)
    X2 = 0.5 * X1 + 10*U
    E = np.random.normal(0,1,n)
    Y = 2 * X1 + 0.3 * X2 + E
    return X1, X2, Y
```

In [3]:
```
X1, X2, Y = generate_dataset(seed=42)

n = len(Y)
d = 2

X = np.column_stack((np.ones(n), X1, X2))
model = LinearRegression(fit_intercept=False)
model.fit(X, Y)

model.coef_
```

Out[3]:
```
array([-0.08939 ,  1.622309,  0.371931])
```

In [4]:
```
mse = np.sum((Y - model.predict(X))**2) / (n - d - 1)
SEs = np.sqrt(mse * np.diag(np.linalg.inv(X.T @ X)))
t_stats = model.coef_ / SEs

2 * (1 - stats.t.cdf(np.abs(t_stats), df=n-d-1))
```

Out[4]:
```
array([0.725864, 0.000004, 0.      ])
```

In [5]:
```
def generate_coefs(n=100, m=1000, rand=False, gen=generate_dataset):
    coefs = []
    for i in range(m):
        X1, X2, Y = gen(n=n, seed=None if rand else i)
        X = np.column_stack((np.ones(len(Y)), X1, X2))
        model = LinearRegression(fit_intercept=False)
        model.fit(X, Y)
        coefs.append(model.coef_)
    return np.array(coefs)
```

In [6]:
```
coefs = generate_coefs(n=100, m=1000)
fig = px.scatter(x=coefs[:, 1], y=coefs[:, 2])
fig.update_layout(
    xaxis_title="w1",
    yaxis_title="w2",
    title="Joint Distribution of Estimated Coefficients"
)
fig.show()
```

In [7]:
```
def reject_null(alpha=0.05, null_val=0, print_coefs=False, **kwargs):
    coefs = generate_coefs(**kwargs)
    means = np.mean(coefs, axis=0)
    stds = np.std(coefs, axis=0)
    radius = stats.norm.ppf(1 - alpha / 2) * stds
    if print_coefs:
        print(np.array([means - radius, means + radius]).T)
    return np.where(np.abs(means - null_val) > radius, 1, 0)
```

In [8]:
```
reject_null(m=1000, print_coefs=True)
```

```
[[-0.542151  0.527369]
 [ 1.31597   2.682614]
 [ 0.23085   0.371991]]
```

Out[8]:
```
array([0, 1, 1])
```

In [9]:
```
rejections = np.array([reject_null(rand=True) for i in range(100)])
rejections.mean(axis=0)
```

Out[9]:
```
array([0., 1., 1.])
```

In [10]:
```
def wide_error(n = 100, seed = 140):
    np.random.seed(seed)
    X1 = np.random.uniform(0,1,n)
    U = np.random.uniform(0,1,n)
    X2 = 0.5 * X1 + 10*U
```

```
    E = np.random.normal(1,10000,n) # <--- wide error
    Y = 2 * X1 + 0.3 * X2 + E
    return X1, X2, Y
```

In [11]:
```
rejections = np.array([reject_null(rand=True, gen=wide_error) for i in range(100)])
rejections.mean(axis=0)
```

Out[11]:
```
array([0., 0., 0.])
```

In [12]:
```
def not_full_rank(n = 100, seed = 140):
    np.random.seed(seed)
    X1 = np.random.uniform(0,1,n)
    X2 = 5 * (1 - X1) # <--- not full rank
    E = np.random.normal(0,1,n)
    Y = 2 * X1 + 0.3 * X2 + E
    return X1, X2, Y
```

In [13]:
```
rejections = np.array([reject_null(rand=True, gen=not_full_rank) for i in range(100)])
rejections.mean(axis=0)
```

Out[13]:
```
array([1.  , 1.  , 0.03])
```

In [14]:
```
coefs = generate_coefs(n=100, m=1000, gen=not_full_rank)
fig = px.scatter(x=coefs[:, 1], y=coefs[:, 2])
fig.update_layout(
    xaxis_title="w1",
    yaxis_title="w2",
    title="Joint Distribution of Estimated Coefficients"
)
fig.show()
```

In [15]:
```
def not_homoskedastic(n = 100, seed = 140):
    np.random.seed(seed)
    X1 = np.random.uniform(0,1,n)
    U = np.random.uniform(0,1,n)
    X2 = 0.5 * X1 + 10*U
    E = np.random.normal(0,1,n)
    Y = 2 * X1 + 0.3 * X2 + E / X1 # <--- not homoskedastic
    return X1, X2, Y
```

In [16]:
```
rejections = [reject_null(rand=True, gen=not_homoskedastic) for i in range(100)]
np.array(rejections).mean(axis=0)
```

Out[16]:
```
array([0., 0., 0.])
```

Problem 4

In [17]:
```
import pandas as pd
import statsmodels.api as sm
df = pd.read_csv("chol_data.csv")
X = df[["Age", "Diabetes"]]
X = np.column_stack((np.ones(len(X)), X))
y = df["TotChol"]
model = sm.OLS(y, X).fit()
print(model.summary())
```

```
                            OLS Regression Results
==============================================================================
Dep. Variable:                TotChol   R-squared:                       0.017
Model:                            OLS   Adj. R-squared:                  0.013
Method:                 Least Squares   F-statistic:                     4.054
Date:                Wed, 22 Oct 2025   Prob (F-statistic):             0.0180
Time:                        18:18:16   Log-Likelihood:                -700.91
No. Observations:                 473   AIC:                             1408.
Df Residuals:                     470   BIC:                             1420.
Df Model:                           2
Covariance Type:            nonrobust
==============================================================================
                 coef    std err          t      P>|t|      [0.025      0.975]
------------------------------------------------------------------------------
const          4.8000      0.156     30.750      0.000       4.493       5.107
x1             0.0075      0.003      2.474      0.014       0.002       0.013
x2            -0.3177      0.161     -1.977      0.049      -0.633      -0.002
==============================================================================
Omnibus:                       34.025   Durbin-Watson:                   2.040
Prob(Omnibus):                  0.000   Jarque-Bera (JB):               42.514
Skew:                           0.603   Prob(JB):                     5.87e-10
Kurtosis:                       3.837   Cond. No.                         181.
==============================================================================

Notes:
[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.
```

In [18]:
```
fig = go.Figure()

# Add scatter points
fig.add_trace(go.Scatter(x=df[df["Diabetes"]==0]["Age"], y=df[df["Diabetes"]==0]["TotChol"],
                         mode='markers', name='No Diabetes', marker=dict(color='blue', opacity=0.6)))
fig.add_trace(go.Scatter(x=df[df["Diabetes"]==1]["Age"], y=df[df["Diabetes"]==1]["TotChol"],
                         mode='markers', name='Diabetes', marker=dict(color='red', opacity=0.6)))

# Add fitted lines
age_range = np.linspace(df["Age"].min(), df["Age"].max(), 200)

# No diabetes line
X_no_diabetes = np.column_stack((np.ones(len(age_range)), age_range, np.zeros(len(age_range))))
y_no_diabetes = model.predict(X_no_diabetes)
fig.add_trace(go.Scatter(x=age_range, y=y_no_diabetes, mode="lines", line=dict(color="blue"), name="No Diabetes Line"))

# Diabetes line
X_diabetes = np.column_stack((np.ones(len(age_range)), age_range, np.ones(len(age_range))))
y_diabetes = model.predict(X_diabetes)
fig.add_trace(go.Scatter(x=age_range, y=y_diabetes, mode="lines", line=dict(color="red"), name="Diabetes Line"))

fig.update_layout(xaxis_title="Age", yaxis_title="Total Cholesterol", title="Fitted Lines by Diabetes Status")
fig.show()
```

In [19]:
```
def plot_ci_pred(data, title, diabetes):
    data["Diabetes"] = data["Diabetes"].astype(str)
    fig = px.scatter(data, x="Age", y="TotChol", color="Diabetes")

    age_grid = np.linspace(data["Age"].min(), data["Age"].max(), 200)
    diabetes_term = np.ones(len(age_grid)) if diabetes else np.zeros(len(age_grid))
    Xg = np.column_stack((np.ones(len(age_grid)), age_grid, diabetes_term))
    fig.add_trace(go.Scatter(x=age_grid, y=model.predict(Xg), mode="lines", line=dict(color="blue"), name="Fitted Line"))
    sf = model.get_prediction(Xg).summary_frame(alpha=0.05)

    fig.add_trace(go.Scatter(x=age_grid, y=sf["mean_ci_lower"], mode="lines",
                             line=dict(color="lightblue", width=0), showlegend=False, hoverinfo="skip"))
    fig.add_trace(go.Scatter(x=age_grid, y=sf["mean_ci_upper"], mode="lines",
                             fill="tonexty", fillcolor="rgba(173,216,230,0.3)",
                             name="95% Confidence Interval", line=dict(color="lightblue", width=0)))

    fig.add_trace(go.Scatter(x=age_grid, y=sf["obs_ci_lower"], mode="lines",
                             line=dict(color="lightcoral", width=0, dash="dash"),
                             showlegend=False, hoverinfo="skip"))
    fig.add_trace(go.Scatter(x=age_grid, y=sf["obs_ci_upper"], mode="lines",
                             fill="tonexty", fillcolor="rgba(240,128,128,0.2)",
                             name="95% Prediction Interval", line=dict(color="lightcoral", width=0, dash="dash")))

    fig.update_layout(xaxis_title="Age", yaxis_title="Total Cholesterol", title=title)
    fig.show()

plot_ci_pred(df[df["Diabetes"] == 1], "Confidence and Prediction Intervals for Individuals with Diabetes", 1)
plot_ci_pred(df[df["Diabetes"] == 0], "Confidence and Prediction Intervals for Individuals without Diabetes", 0)
```

```
C:\Users\issac\AppData\Local\Temp\ipykernel_26444\3397294619.py:2: SettingWithCopyWarning:


A value is trying to be set on a copy of a slice from a DataFrame.
Try using .loc[row_indexer,col_indexer] = value instead

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy

C:\Users\issac\AppData\Local\Temp\ipykernel_26444\3397294619.py:2: SettingWithCopyWarning:


A value is trying to be set on a copy of a slice from a DataFrame.
Try using .loc[row_indexer,col_indexer] = value instead

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy
```

In [20]:
```
age_45 = np.array([1, 45, 1])
model.get_prediction(age_45).summary_frame(alpha=0.05)
```

Out[20]:

	mean	mean_se	mean_ci_lower	mean_ci_upper	obs_ci_lower	obs_ci_upper
0	4.819477	0.155226	4.514454	5.124499	2.698189	6.940764

In [21]:
```
age_45 = np.array([1, 45, 0])
model.get_prediction(age_45).summary_frame(alpha=0.05)
```

Out[21]:

	mean	mean_se	mean_ci_lower	mean_ci_upper	obs_ci_lower	obs_ci_upper
0	5.137143	0.053254	5.032498	5.241787	3.035293	7.238992

In [22]:
```
def plot_residuals(data, title, index):
    residuals = model.resid[index]
    fig = px.scatter(x=data["Age"], y=residuals)
    fig.add_trace(go.Scatter(x=data["Age"], y=np.zeros(len(data["Age"])), mode="lines",
                             line=dict(color="black", width=1, dash="dash"),
                             showlegend=False, hoverinfo="skip"))
    fig.update_layout(
        xaxis_title="Age",
        yaxis_title="Residuals",
        title=title
    )
    fig.show()

plot_residuals(df[df["Diabetes"] == 1], "Residuals vs Fitted Values for Individuals with Diabetes", df["Diabetes"] == 1)
plot_residuals(df[df["Diabetes"] == 0], "Residuals vs Fitted Values for Individuals without Diabetes", df["Diabetes"] == 0)
```

In [23]:
```
df["Diabetes"] = df["Diabetes"].astype(int)
df["agediabetes"] = df["Age"] * df["Diabetes"]
X = df[["Age", "Diabetes", "agediabetes"]]
X = sm.add_constant(X)
y = df["TotChol"]
model = sm.OLS(y, X).fit()
print(model.summary())
```

```
                            OLS Regression Results
==============================================================================
Dep. Variable:                TotChol   R-squared:                       0.032
Model:                            OLS   Adj. R-squared:                  0.026
Method:                 Least Squares   F-statistic:                     5.216
Date:                Wed, 22 Oct 2025   Prob (F-statistic):            0.00150
Time:                        18:18:16   Log-Likelihood:                -697.19
No. Observations:                 473   AIC:                             1402.
Df Residuals:                     469   BIC:                             1419.
Df Model:                           3
Covariance Type:            nonrobust
==============================================================================
                 coef    std err          t      P>|t|      [0.025      0.975]
------------------------------------------------------------------------------
const          4.6957      0.160     29.405      0.000       4.382       5.010
Age            0.0096      0.003      3.101      0.002       0.004       0.016
Diabetes       1.7187      0.764      2.250      0.025       0.218       3.220
agediabetes   -0.0335      0.012     -2.726      0.007      -0.058      -0.009
==============================================================================
Omnibus:                       34.285   Durbin-Watson:                   2.048
Prob(Omnibus):                  0.000   Jarque-Bera (JB):               43.283
Skew:                           0.601   Prob(JB):                     3.99e-10
Kurtosis:                       3.866   Cond. No.                         840.
==============================================================================

Notes:
[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.
```

In [24]:
```
fig = go.Figure()

# Add scatter points
fig.add_trace(go.Scatter(x=df[df["Diabetes"]==0]["Age"], y=df[df["Diabetes"]==0]["TotChol"],
                         mode='markers', name='No Diabetes', marker=dict(color='blue', opacity=0.6)))
fig.add_trace(go.Scatter(x=df[df["Diabetes"]==1]["Age"], y=df[df["Diabetes"]==1]["TotChol"],
                         mode='markers', name='Diabetes', marker=dict(color='red', opacity=0.6)))

# Add fitted lines
age_range = np.linspace(df["Age"].min(), df["Age"].max(), 200)

# No diabetes line
X_no_diabetes = np.column_stack((np.ones(len(age_range)), age_range, np.zeros(len(age_range)), np.zeros(len(age_range))))
y_no_diabetes = model.predict(X_no_diabetes)
fig.add_trace(go.Scatter(x=age_range, y=y_no_diabetes, mode="lines", line=dict(color="blue"), name="No Diabetes Line"))

# Diabetes line
X_diabetes = np.column_stack((np.ones(len(age_range)), age_range, np.ones(len(age_range)), age_range))
y_diabetes = model.predict(X_diabetes)
fig.add_trace(go.Scatter(x=age_range, y=y_diabetes, mode="lines", line=dict(color="red"), name="Diabetes Line"))

fig.update_layout(xaxis_title="Age", yaxis_title="Total Cholesterol", title="Fitted Lines by Diabetes Status")
fig.show()
```

In [25]:
```
def plot_ci_pred(data, title, diabetes):
    data["Diabetes"] = data["Diabetes"].astype(str)
    fig = px.scatter(data, x="Age", y="TotChol", color="Diabetes")

    age_grid = np.linspace(data["Age"].min(), data["Age"].max(), 200)
```

```
    diabetes_term = np.ones(len(age_grid)) if diabetes else np.zeros(len(age_grid))
    Xg = np.column_stack((np.ones(len(age_grid)), age_grid, diabetes_term, age_grid * diabetes_term))
    fig.add_trace(go.Scatter(x=age_grid, y=model.predict(Xg), mode="lines", line=dict(color="blue"), name="Fitted Line"))
    sf = model.get_prediction(Xg).summary_frame(alpha=0.05)

    fig.add_trace(go.Scatter(x=age_grid, y=sf["mean_ci_lower"], mode="lines",
                             line=dict(color="lightblue", width=0), showlegend=False, hoverinfo="skip"))
    fig.add_trace(go.Scatter(x=age_grid, y=sf["mean_ci_upper"], mode="lines",
                             fill="tonexty", fillcolor="rgba(173,216,230,0.3)",
                             name="95% Confidence Interval", line=dict(color="lightblue", width=0)))

    fig.add_trace(go.Scatter(x=age_grid, y=sf["obs_ci_lower"], mode="lines",
                             line=dict(color="lightcoral", width=0, dash="dash"),
                             showlegend=False, hoverinfo="skip"))
    fig.add_trace(go.Scatter(x=age_grid, y=sf["obs_ci_upper"], mode="lines",
                             fill="tonexty", fillcolor="rgba(240,128,128,0.2)",
                             name="95% Prediction Interval", line=dict(color="lightcoral", width=0, dash="dash")))

    fig.update_layout(xaxis_title="Age", yaxis_title="Total Cholesterol", title=title)
    fig.show()

plot_ci_pred(df[df["Diabetes"] == 1], "Confidence and Prediction Intervals for Individuals with Diabetes", 1)
plot_ci_pred(df[df["Diabetes"] == 0], "Confidence and Prediction Intervals for Individuals without Diabetes", 0)
```

```
C:\Users\issac\AppData\Local\Temp\ipykernel_26444\2040518627.py:2: SettingWithCopyWarning:


A value is trying to be set on a copy of a slice from a DataFrame.
Try using .loc[row_indexer,col_indexer] = value instead

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy

C:\Users\issac\AppData\Local\Temp\ipykernel_26444\2040518627.py:2: SettingWithCopyWarning:


A value is trying to be set on a copy of a slice from a DataFrame.
Try using .loc[row_indexer,col_indexer] = value instead

See the caveats in the documentation: https://pandas.pydata.org/pandas-docs/stable/user_guide/indexing.html#returning-a-view-versus-a-copy
```

In [26]:
```
age_45 = np.array([1, 45, 1, 45])
model.get_prediction(age_45).summary_frame(alpha=0.05)
```

Out[26]:

	mean	mean_se	mean_ci_lower	mean_ci_upper	obs_ci_lower	obs_ci_upper
0	5.342805	0.246226	4.858962	5.826647	3.202363	7.483247

In [27]:
```
age_45 = np.array([1, 45, 0, 0])
model.get_prediction(age_45).summary_frame(alpha=0.05)
```

Out[27]:

	mean	mean_se	mean_ci_lower	mean_ci_upper	obs_ci_lower	obs_ci_upper
0	5.129421	0.052969	5.025335	5.233506	3.041785	7.217056

In [28]:
```
plot_residuals(df[df["Diabetes"] == 1], "Residuals vs Fitted Values for Individuals with Diabetes", df["Diabetes"] == 1)
plot_residuals(df[df["Diabetes"] == 0], "Residuals vs Fitted Values for Individuals without Diabetes", df["Diabetes"] == 0)
```